A Method to Construct Counterexamples for Greedy Algorithms

Jagadish M.[1] Sridhar Iyer[2]
Indian Institute of Technology Bombay

'Greedy algorithms' is a prominent topic in any undergraduate algorithms course. Most textbooks discuss a method called *exchange argument* that is helpful in the analysis of greedy algorithms. However, the exchange argument helps only in proving the correctness of a greedy strategy. It is not useful in disproving an incorrect greedy strategy. A common way to disprove a strategy is to construct an input instance on which the strategy fails. Such an instance is called a *counterexample*. We give a method to construct counterexamples for greedy algorithms. We show that our method works on many problems in graph theory. We did a pilot experiment with students and found that the key idea in our approach did not occur to students.

1.1 Introduction

Counterexamples often crop up in the context of greedy algorithms. For example, every third exercise problem in the chapter on greedy algorithms in the Kleinberg-Tardos textbook [5] requests for a proof or a counterexample. Broadly, the task of constructing counterexamples is as follows. We are given an optimization problem P and a greedy algorithm that is purported to solve P. We need to construct an instance of P on which the greedy algorithm fails (Fig. 1.1). We present a technique called the *Anchor method* that can be used to construct counterexamples. We limit the scope of our work to simple greedy approaches and the scope of our problems to graph theoretic problems on unweighted graphs. We show that the anchor method works on many well-known problems. The specific list of problems can be found in Table 1.2.

Remark. Preliminary version of this work appears in [6].

[1]Email: jagadish@cse.iitb.ac.in
[2]Email: sri@iitb.ac.in

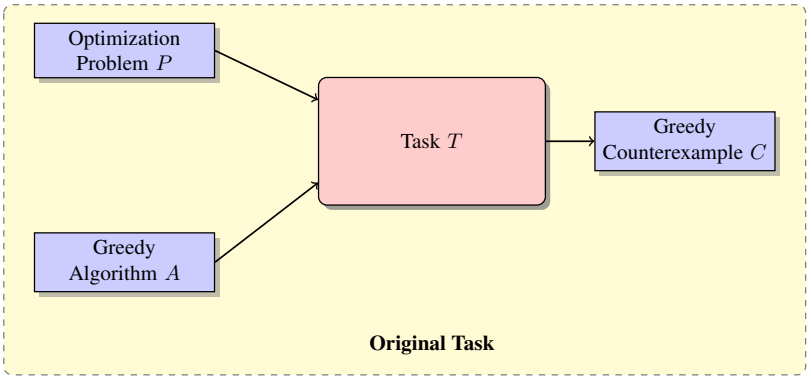


Figure 1.1: A greedy counterexample is an instance of P on which the greedy algorithm fails.

1.2 Definitions

In this section, we prepare the ground by first giving a few definitions.

1.2.1 Definitions related to graphs

A *graph* is an ordered pair $G = (V, E)$ where V is the set of *vertices* and E the set of pairs of vertices called *edges*. The pair of vertices in an edge are unordered so edge (u, v) is same as (v, u). For an edge $e = (u, v)$ we say e is incident on vertex u and v. Vertices u and v are also called *endpoints* of e. The *degree* of a vertex is the number of edges incident on it.

A *subgraph* of G is a graph H such that $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$ and the assignment of endpoints to edges in H is the same as in G. A graph H is a *proper subgraph* of G if H is subgraph of G and $H \neq G$.

We *delete* a vertex v from $G = (V, E)$ by removing v from V and removing all the edges that are incident on v from E.

1.2.2 Definitions related to problems

In a *constraint problem* P, we are given a graph G and a set of constraints as input. A *solution* to the constraint problem P is a subgraph of G that satisfies the given set of constraints.

An *optimization problem* is a constraint problem in which each solution is assigned a *value* (a number) and the objective is to find a solution with the optimal value OPT. We call the optimization problem as a minimization or maximization problem, accordingly.

MIS Problem. Let $G = (V, E)$ be a graph. An independent set in G is a vertex set $S \subseteq V$ such that no two vertices in S have an edge between them in E. The problem is to find an

independent set in G with maximum number of vertices. See Fig. 1.2.

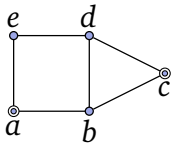


Figure 1.2: In the graph shown, $S = \{a, c\}$ is a maximum independent set (MIS). So, OPT=2.

Every independent set in G is a solution to the problem. The value of a solution is the number of vertices in it. The MIS problem is therefore an optimization problem.

1.2.3 Definitions related to algorithms

An *algorithm* for an optimization problem is a well-defined procedure which finds a solution to the problem (not necessarily the optimal).

Greedy algorithm. The greedy algorithm works by 'making the choice that looks best at the moment' [5]. We do not dwell on what exactly qualifies as a greedy algorithm. The notion of locally-best choice will appeal only intuitively.

Greedy algorithm for MIS. Consider the following greedy algorithm to solve the MIS problem. When a vertex is picked, we are prohibited from picking its neighbors. Since we want to pick as many vertices as possible, intuitively it is reasonable to pick a vertex with least degree first [7].

Algorithm 1.2.1: MIS(G)

Input: Graph G

Output: An independent set S from graph G

Initially the set S is empty.

1 : Pick a vertex v from G with the least degree.

Add v to S.

2 : Delete v and all its neighbors from G.

3 : Repeat the above two steps until G has no vertices.

4 : Report S as the solution

Weak algorithm. We define a weak algorithm to be an algorithm that works by making an *arbitrary* choice at each moment.

1.2.4 Definitions related to counterexamples

Counterexample. For a given optimization problem and an algorithm, a *counterexample* is a problem instance on which the algorithm produces a non-optimal solution.

Plausible and definitive counterexamples. If more than one choice looks best at a given moment, the greedy algorithm picks one of the best choices arbitrarily. Hence, an algorithm could have multiple execution paths for the same input instance. We call an input instance a *plausible counterexample* if one of the execution paths leads to a wrong answer. Similarly, an input instance is called a *definitive counterexample* if every possible path of execution leads to a wrong answer. It is a convention to treat plausible counterexamples as valid counterexamples. So for the rest of the chapter, we refer to plausible counterexamples simply as counterexamples.

Weak and greedy counterexamples. A *weak counterexample* is a counterexample to a weak algorithm. Similarly, a *greedy counterexample* is a counterexample to a greedy algorithm.

Goodness ratio. The *goodness ratio* of a counterexample is a measure that compares the value of the optimal solution to the value of the solution obtained by the algorithm. We define it as follows: Suppose I is an input to an algorithm A. Let OPT be the value of the optimal solution to I and ALG be the value of the solution returned by the algorithm A. The goodness ratio of the input I is defined as the ratio of OPT to ALG, for maximization problems. For minimization problems, it is the ratio of ALG to OPT. For example, for the input shown in Fig. 1.8 OPT=3 and ALG=2, so the goodness ratio is 1.5.

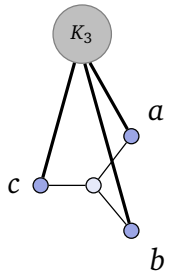


Figure 1.3: A counterexample to Alg. 1.2.1 for the MIS problem. A dark edge indicates that there are edges from the high degree vertex to all the three vertices in K_3. The central vertex has degree 3. Vertices a, b and c have degree 4 each. Each vertex in K_3 has degree 5. So the greedy algorithm picks the central vertex first and then picks one vertex from K_3. The maximum independent set is actually $\{a, b, c\}$. Hence, the greedy algorithm picks 2 vertices, while the optimal algorithm picks 3.

We usually express goodness ratio in an asymptotic sense *i.e.* as a function of the number of vertices in the counterexample.

General Counterexample. A *general counterexample* is a graph family of counterexamples such that for any given number k, we can find a graph in the family with larger than k vertices. A general counterexample is said to be *tight* if it achieves the best possible goodness ratio (in the asymptotic sense).

1.2.5 Definitions related to solutions

Local solution. Let G be the input graph of a maximization problem P. A solution to P is said to be *local* if it is not a proper subgraph of any other solution in G. Likewise, a solution to a minimization problem P' involving input graph G' is said to be local if no proper subgraph of it is a solution in G'.

Discrepancy. Suppose G is the input graph of an optimization problem. Let S_1 and S_2 be two local solutions in G whose values are s_1 and s_2, respectively. Without loss of generality, assume that $s_1 >= s_2$. The ratio s_1/s_2 is called the *discrepancy* of the graph G.

1.3 Anchor Method

The Anchor method consists of three main steps which are briefly described below.

Step 1. Construct a counter-example for the weak algorithm.

(Tip: Try graphs with high discrepancy.)

Step 2. Observe how the weak algorithm fails. Capture the essence of the weak counterexample and call it an *anchor*.

Step 3. Extend the weak counterexample to a greedy counterexample.

(Tip: Try attaching graphs with low discrepancy.)

1.4 Anchor Method using MIS as an Illustrative Example.

Step 1 Analyze the task

The scope of the method is limited to graph-theoretic optimization problems and greedy algorithms. We make sure that the given problem is within the scope by verifying if the

given problem is an optimization problem as defined in Sec. 1.2.2.

The attributes of the MIS problem are as follows.

- *Input.* A graph $G = (V, E)$.
- *Solution.* A set of vertices $S \subseteq V$. Hence, the solution is a sub-graph of G as required.
- *Constraints.* A set of vertices S must be independent.
- *Value of a solution.* The number of vertices in S.
- *Greedy choice.* Pick the least-degree vertex first.

Clearly, MIS problem is an optimization problem and belongs to the scope of problems for which the method is applicable.

Task. To construct a counterexample to the greedy algorithm for the MIS problem.

Step 2 Weakening Step

Our objective is to construct a counterexample to a given greedy algorithm. We first ask if we can achieve a simpler objective: Can we construct a counterexample for an algorithm that does not make any greedy choices?

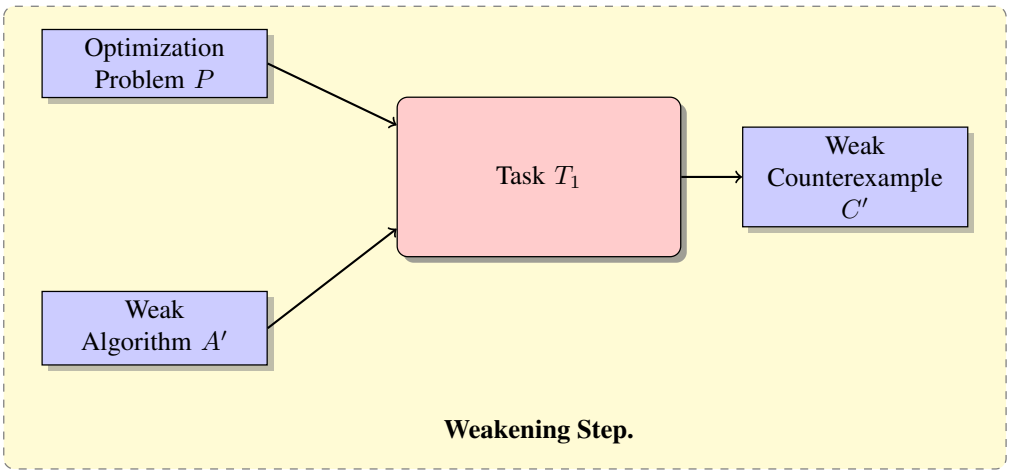


Figure 1.4: Instead of solving the original task directly, we first pose an easier task. The optimization problem is kept the same but the algorithm is changed from greedy to a weak algorithm.

Weak Algorithm. The greedy algorithm works by 'making the choice that looks best at the moment' [5]. We define a weak algorithm to be an algorithm that works by making an *arbitrary* choice at each moment.

We call the counterexample to the weak algorithm as a *weak counterexample*. Recall that a weak algorithm is obtained from the greedy algorithm by replacing the greedy choice with an arbitrary choice. The assumption is that the weak counterexample gives insight into the structure of the greedy counterexample.

The algorithm Alg. 1.2.1 picks the vertices in increasing order of degree, so the corresponding weak algorithm picks vertices in arbitrary order.

So now the weaker task is to construct a counterexample for the following algorithm:

Algorithm 1.4.1: MIS-WEAK(G)

Input: Graph G

Output: An independent set S from graph G

Initially the set S is empty.

1 : Pick a vertex v from G.

Add v to S.

2 : Delete v and all its neighbors from G.

3 : Repeat the above two steps until G has no vertices.

4 : Report S as the answer

Weak Counterexample. A weak counterexample is an instance on which the weak algorithm fails assuming the worst possible execution or, in other words, when choices are made in an adversarial manner (e.g. Fig. 1.5).

Step 3 Solving the candidate problem

In this step, we construct a weak counterexample. There could be several weak counterexamples for a given weak algorithm. It helps to construct multiple weak counterexample with high discrepancy.

A counterexample to Alg. 1.4.1 is shown in Fig. 1.5.

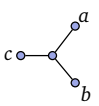


Figure 1.5: A counterexample to the weak algorithm for the MIS problem. A star graph with three outer vertices.

The weak algorithm could pick the central vertex first and get an independent set of size 1, whereas the maximum independent set has size 3 ($\{a, b, c\}$). So this is our weak counterexample.

In Sec. 1.5, we explain how extremality can be used to pick the 'good' weak counterexamples.

Step 4 **Extending the weak counterexample**

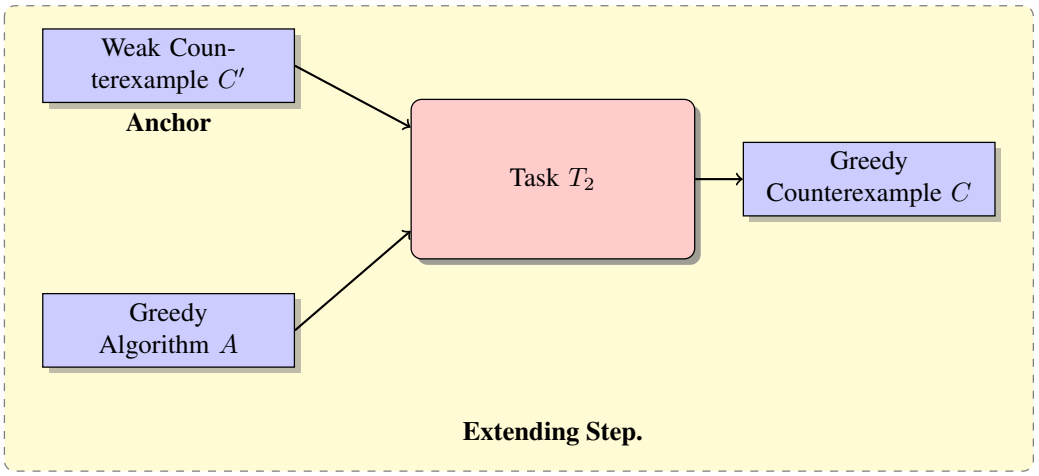


Figure 1.6: In the extend step, we use the weak counterexample from the previous step to guide the process of constructing a greedy counterexample.

In this step, we extend the weak counterexample to a greedy counterexample (Fig. 1.6). We do so following two steps:

1. First, we observe how the weak algorithm behaves on the weak counterexample. Then we ask ourselves, what would have to happen if the greedy algorithm were to go wrong in the same way as the weak algorithm? We isolate this 'bad structure' and call it the *anchor*.

2. We attach a graph to the anchor to obtain a greedy counterexample. The attached graph is called the *auxiliary structure*. The term is meant to suggest that this structure performs a supporting role to the anchor.

In Sec. 1.5, we explain how extremality can be helpful in picking the right auxiliary structure.

Building an anchor for the MIS problem. In the weak counterexample (Fig. 1.5), the central vertex has the highest degree. We want the greedy algorithm to go wrong in the

same way as the weak algorithm. We know that the greedy algorithm would pick the central vertex in the above graph first if the central vertex had the *lowest* degree *somehow*. We capture this intuition in the anchor (Fig. 1.7). The anchor is a skeleton that captures the essence of a possible greedy counterexample. It serves as a starting point for constructing a counterexample to the greedy algorithm.

Notation. We use symbols ● and ○ to indicate a vertex we *wish* were of high degree and low degree, respectively.

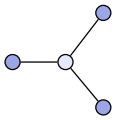


Figure 1.7: The anchor obtained from the weak counterexample. If the central degree had the lowest degree somehow, then the greedy algorithm would fail in the same way as the weak algorithm.

Adding an auxiliary structure. Each high degree vertex needs to be connected to at least 3 vertices in order to exceed the low degree central vertex. Let K_n denote a complete graph with n vertices. Attaching the auxiliary graph K_3 gives a counterexample as shown in Fig. 1.8.

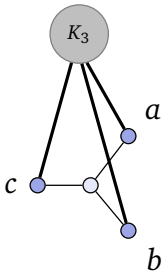


Figure 1.8: A counterexample to Alg. 1.2.1 for the MIS problem. A dark edge indicates that there are edges from the high degree vertex to all the three vertices in K_3. The central vertex has degree 3. Vertices a, b and c have degree 4 each. Each vertex in K_3 has degree 5. So the greedy algorithm picks the central vertex first and then picks one vertex from K_3. The maximum independent set is actually $\{a, b, c\}$. Hence, the greedy algorithm picks 2 vertices, while the optimal algorithm picks 3.

1.5 Role of Extremality

In this section, we explain how to pick useful weak counterexamples and auxiliary structures based on the principle of extremality. We may have to try multiple weak counterexam-

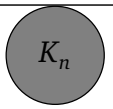
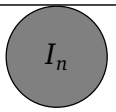
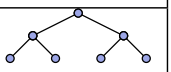
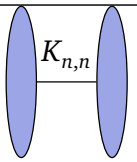
Common Extremal Graphs	
Path	
Cycle	
Star	
Complete graph	
Independent set graph	
Binary Tree	
Complete bipartite graph	

Table 1.1: High or low discrepancy graphs usually come from simple graphs shown above.

ples and auxiliary structures before hitting upon the right combination that gives a greedy counterexample. The following tips are helpful in picking the right structures:

Tip for choosing the weak counterexample. Pick those graphs which have the highest discrepancy.

Tip for choosing the auxiliary structure. Pick those graphs which have the least discrepancy.

Usually, the extremal graphs that have the highest or the least discrepancy are easy to find by trail and error. They are usually composed of simple structures like the ones shown in Table 1.1.

Let us apply this tip to picking a weak counterexample for the MIS problem. We want to choose such a graph which has two local solutions of different size. Searching through the generic extremal graphs (Table 1.1) we find that the star graph has this property.

Let us pick an auxiliary structure for the MIS problem based on discrepancy.

Attaching the auxiliary structure K_n to the anchor gives us a general counterexample (Fig. 1.11). This counterexample has a goodness ratio of $O(n)$ which is also tight.

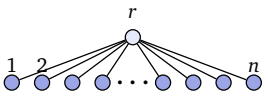


Figure 1.9: A weak counterexample and the corresponding anchor. There are two local solutions in the counterexample, one of size 1 and the other of size n. The discrepancy is therefore $O(n)$, which is the highest possible discrepancy for the MIS problem.



Figure 1.10: Two graphs with discrepancy 1. In the complete graph (a), any local solution has size 1. In an independent set graph (b) any local solution has size n. Hence, K_n and I_n are extremal graphs with the least discrepancy for the MIS problem. Again, both these graphs were among the generic extremal graphs shown in Table 1.1.

1.6 Additional Examples

We show the applicability of the anchor method on many classic problems from graph theory. Most of the problem descriptions along with their applications can be found in standard texts like [2, 8].

1.6.1 Minimum Vertex Cover Problem

Optimization Problem. Let $G = (V, E)$ be a graph. A vertex cover problem is a vertex set $S \subseteq V$ such that for every edge $(u, v) \in E$ either vertex u or v (or both) belongs to S. The vertex cover problem is to find a vertex cover of minimum size. An example is shown in Fig. 1.12.

Greedy Algorithm. A vertex that has many edges is probably a good pick. Let $v_{max}(G)$

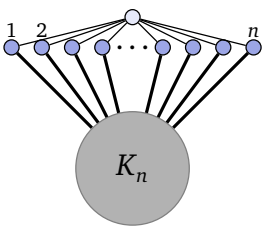


Figure 1.11: A general counterexample to Alg. 1.2.1 for the MIS problem with $O(n)$ goodness ratio. The greedy algorithm picks 2 independent vertices, while the optimal algorithm picks n vertices.

Problem	Anchor	Aux. St.	G. Ratio	Best Ratio
INDEPENDENT SET	Star	K_n	$O(n)$	$O(n)$
VERTEX COVER	Star	Centipede	2	$\log n$
MATCHING	Paths	$K_{n,n}$	2	2
MAXLEAF	Path	Binary tree	2	2
MAXCUT	$K_{n,n}$	$K_{n,n}$	2	2
NETWORK FLOW	-	Paths	$O(\sqrt{n})$	$O(\sqrt{n})$
FEEDBACK SET	-	K_ns	1.5	$O(\log n)$
DOMINATING SET	Paths	Paths	1.5	$O(\log n)$

Table 1.2: Applicability of anchor method on common graph-theoretic problems. The anchors and the auxiliary structures of most counterexamples come from simple extremal graphs (Table 1.1). Symbol - indicates that the anchor is problem-specific. Descriptions of the problems and the corresponding greedy algorithms can be found in Sec. 1.6.

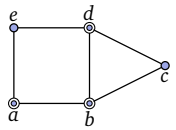


Figure 1.12: In the graph shown, $S = \{a, b, d\}$ is a minimum vertex cover.

denote a vertex with maximum degree in graph G.

Algorithm 1.6.1: MVC(G)

Input: Graph G

Output: A vertex cover set S of graph G

1 : **if** G has at least one edge

add vertex $v_{max}(G)$ to set S.

else report S as the answer

2 : Remove vertex $v_{max}(G)$ from G to get G'.

3 : Recurse with G' as the input: MVC(G').

Constructing a counterexample

1 **Weak algorithm.** The greedy algorithm picks the vertex with highest degree at each step. A weak algorithm picks an arbitrary vertex at each step.

2 **Weak counterexample.** A star graph is a simple extremal graph on which the weak algorithm fails. There are two local solutions in the star graph (Fig. 1.13):

- S_1 : The root node r.
- S_2 : The vertices labelled 1 to n.

The weak algorithm could pick the leaf nodes and find solution S_2, whereas S_1 is the minimum vertex cover.

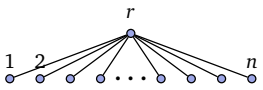


Figure 1.13: A weak counterexample for the vertex cover problem. The discrepancy of the graph is $O(n)$. The star graph also has the highest discrepancy among all the weak counterexamples.

3 **Building an anchor.** To build an anchor we ask ourselves: "What would have to happen if the greedy algorithm were to fail in the same way as the weak algorithm?" The greedy algorithm pick the highest degree vertex first. So if *somehow* the leaf nodes had high degree and the root node had the lowest degree then the greedy algorithm would fail in the same way as the weak algorithm. We capture this intuition by an anchor (Fig. 1.14).

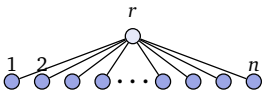


Figure 1.14: An anchor corresponding to the weak counterexample. We wish the root node were of low degree and the leaf nodes were of high degree.

4 **Adding the auxiliary structure.** We need to find an auxiliary structure that can be attached to the anchor. As mentioned in Sec. 1.5, it helps to first look at simple extremal graphs with low discrepancy. Fig. 1.15 shows a few such graphs.

We try to extend the anchor to a greedy counterexample by trying to attach each one of the graphs shown in Fig. 1.15. Attaching a path of length $n+2$ gives a greedy counterexample (Fig. 1.16). Similarly, attaching a centipede of length $n+3$ gives a greedy

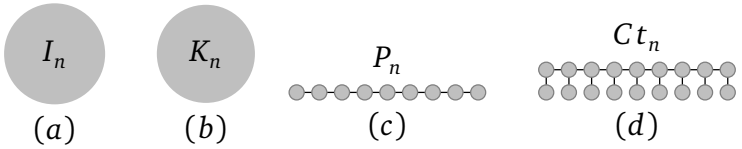


Figure 1.15: Four simple graphs with discrepancy 1. The value of any local solution (vertex cover) in each graph from left to right is 0, $n-1$, $n/2$ and n, respectively.

counter example (Fig. 1.17). For K_n, attaching one K_n makes the vertices in K_n itself as the highest degree vertices so we try attaching two K_ns, which does the trick (Fig. 1.18). There does not seem to be any way of extending by attaching I_n.

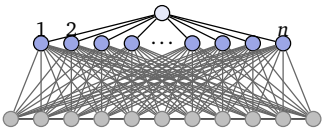


Figure 1.16: A counterexample for vertex cover. The auxiliary path attached has length $n+2$. Each leaf node of the anchor has degree $n+3$. The greedy algorithm picks all the leaf nodes of the anchor and alternate nodes from the path. This gives a vertex cover of size $\approx 1.5n$. The optimal solution is to pick the root node of the anchor and all the vertices in the attached path. This gives a minimum vertex cover of size $n+3$. Hence the goodness ratio of this counterexample is $\lim_{n\to\infty} \frac{1.5n}{n+3} = 1.5$.

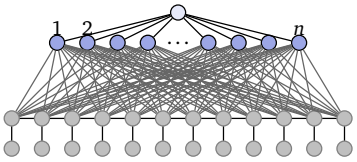


Figure 1.17: A counterexample for vertex cover with goodness ratio 2. The centipede attached has length $n+3$.

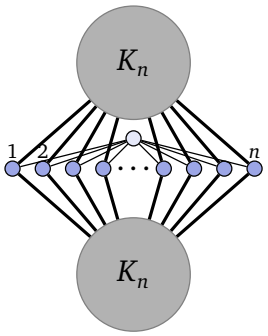


Figure 1.18: A counterexample for vertex cover using K_ns as an auxiliary structure. The goodness ratio of the counterexample is 1.5.

1.6.2 Bipartite Matching

A graph $G = (V, E)$ is *bipartite* if the vertex set can be partitioned into two sets A and B such that no edge in E has both endpoints in A or B alone.

A set of edges M is called a *matching* if $M \subseteq E$ and every vertex in G is an endpoint of at most one edge in M. A vertex is said to be *matched* if it is an endpoint of some edge in M; if not the vertex is said to be *unmatched*.

Optimization Problem. Let $G = (V, E)$ be a bipartite graph. The bipartite matching problem (MBM) is to find a matching M of maximum size in G

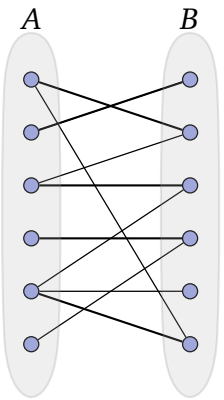


Figure 1.19: In the graph above, the dark edges form a matching of size 5.

Greedy Algorithm. A vertex with degree one has to necessarily be matched to its neighbor if it has to participate in the matching. In general, a vertex with fewer neighbors has fewer choices and therefore more critical.

Notation. Let v_{min} denote the vertex with minimum non-zero degree in graph G and

u_{min} denote the vertex with minimum degree adjacent to v_{min}.

Algorithm 1.6.2: MM(G)

Input: Graph G

Output: A matching M from graph G

Initially the matching set M is empty.

1 : **if** graph G has no edges

report M as the matching set.

2 : Add the edge (v_{min}, u_{min}) to the set M.

3 : Remove the vertices v_{min} and u_{min} from G

to get the subgraph G'.

4 : Recurse on the graph G': MM(G')

Constructing a counterexample

1 **Weak algorithm.** The weak algorithm picks edges in any order and includes them in matching set M.

2 **Weak counterexample.** A path of four nodes in the simplest instance on which the weak algorithm fails. A general counterexample is obtained by making many copies of P_4, as shown in Fig. 1.20. The weak algorithm could pick just the middle edges, while the optimal algorithm picks the first and the last edge of each path. Hence, the optimal solution can be twice as good as the weak algorithm's solution.

3 **Building an anchor.** The greedy algorithm would behave like the weak algorithm if the end points of each path were of high degree. The anchor captures this intuition (Fig. 1.21).

4 **Adding the auxiliary structure.**

We attach an auxiliary structure ($K_{2,2}$) to each of the high degree nodes. If we start with the anchor obtained by the general weak counterexample (Fig. 1.22) we can get a tight counterexample with goodness ratio 2 (Fig. 1.23). Observe that $K_{n,n}$ has the least discrepancy since all the local solutions have size n.

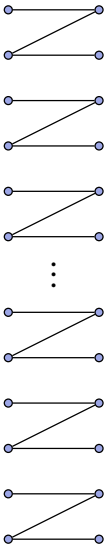


Figure 1.20: A general weak counterexample for the bipartite matching problem with goodness ratio 2.

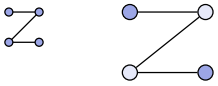


Figure 1.21: The anchor for bipartite matching is obtained by replacing the end vertices of P_4 with high degree vertices. We want the greedy algorithm to pick the middle edge. This would happen if the end vertices were of high degree and the middle nodes were of low degree.

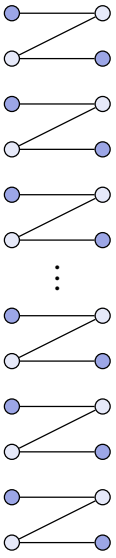


Figure 1.22: The anchor obtained from the general weak counterexample.

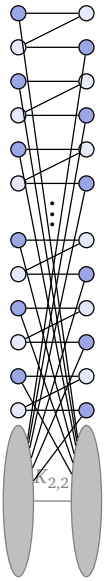


Figure 1.23: A tight counterexample for the bipartite matching problem. The shaded vertices have degree three. The goodness ratio of the graph is 2.

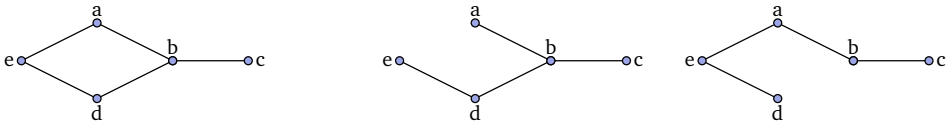


Figure 1.24: Two spanning trees for the graph given on the left: one with 3 leaves and the other with 2 leaves.

1.6.3 Maximum leaf spanning tree

Optimization Problem. Given a graph $G = (V, E)$, the maximum leaf spanning tree problem (MLS) is to find a spanning tree T in G that has maximum number of leaves; where leaves are vertices with degree 1. An example is shown in Fig. 1.24.

Greedy Algorithm. We would like to grow the tree along vertices that branch out well. One strategy is to grow the tree along a vertex which has the maximum number of neighbors not in the tree [4].

Notation. For a vertex v in tree T, let $N'(v)$ denote the set of neighbors of v not in T.

Algorithm 1.6.3: MLS(G)

Input: A connected graph G

Output: A spanning tree T of graph G

Tree T initially has only the vertex $v_{max}(G)$.

repeat

1 : Pick a vertex u in T such that the
size of the set $N'(u)$ is maximized.

2 : Grow the tree T by attaching
vertices in $N'(u)$ to u.

until the tree T has all the vertices in G

Constructing a counterexample

1 **Weak algorithm.** In the greedy algorithm, we grow the tree T along the vertex that has maximum number of neighbors outside the tree. Consider the weak algorithm that does not exercise this choice and instead grows the tree along any arbitrary vertex.

2 **Weak counterexample.** None of the simple extremal graphs is a counterexample, so we need to construct a weak counterexample by trial and error. An instance where the

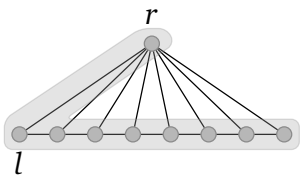


Figure 1.25: A weak counterexample for the max-leaf spanning tree problem. The optimal solution is to start with the node r and pick all its neighbors. The weak algorithm could start with the leftmost node and pick a spanning tree having only two leaves.

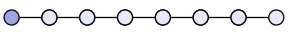


Figure 1.26: If the greedy algorithm has to fail like the weak algorithm then the leftmost node must have high degree and all the nodes of the path must be leaves of some tree.

weak algorithm could perform poorly is given in Fig. 1.25. The optimal algorithm would pick the vertex with the highest degree r and obtain a spanning tree having $|V| - 1$ leaves. If the weak algorithm picks vertex l first, it could end up with a spanning tree having only two leaves (as indicated with dark background).

3 **Building an anchor.**

The key to designing the weak counterexample was to make the algorithm select a long path of vertices which should have been selected as leaves. We want the greedy algorithm to mimic that behaviour? We make the greedy algorithm start from the first node by increasing its degree.

4 **Adding the auxiliary structure.** The vertices of this path must be the leaves of some tree. We need to attach an auxiliary structure such that the vertices on the path become leaves. The auxiliary structure cannot have a vertex with degree greater than 3 since we have forced the greedy algorithm to pick the the leftmost vertex in the anchor first. The complete binary tree serves this purpose. The counterexample is shown in Fig. 1.27.

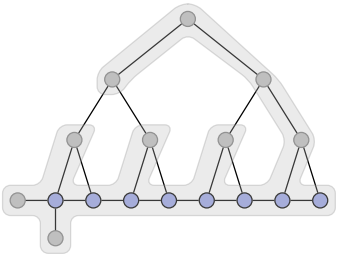


Figure 1.27: A tight counterexample for max-leaf spanning tree problem with goodness ratio 2.

1.6.4 Max Cut

Optimization Problem. Let $G = (V, E)$ be a graph, and $S \subseteq V$ be any subset. Then $E(S, \overline{S}) \stackrel{def}{=} \{(u, v) : u \in S \text{ and } v \notin S\}$ forms a cut. The MAX-CUT problem is as follows. Given a graph G as input, find $S \subseteq V$ which maximizes the value of $|E(S, \overline{S})|$.

Greedy Algorithm. Suppose $E(S, \overline{S})$ is a cut. If moving a vertex from S to $\overline{S}$ increases the value of the cut, then we must do so. A greedy strategy is to move a vertex that results in the largest increase in the value of the cut.

Algorithm 1.6.4: MAXCUT(G)

Input: A graph $G = (V, E)$

Output: A subset $S \subseteq V$

Initialize $S = V$.

repeat

1 : Let $v \in V$ be the vertex that results in
the maximum increase in the value of the cut
if moved from S to $\overline{S}$ (or vice-versa)

2 : Move v from S to $\overline{S}$ (or vice-versa).

until there is no positive increase in the value of
the cut by moving any vertex.

Constructing a counterexample

1 **Weak algorithm.** The greedy algorithm moves a vertex that results in the largest increase in the cut. The weak algorithm moves the vertex that results in the smallest increase in the cut.

2 **Weak counterexample.** There are many generic extremal graphs that are counterexamples to the weak algorithm. We pick $K_{n,n}$ since it has the highest discrepancy. There exists a cut with size n^2 but the weak algorithm settles for a cut with size zero.

3 **Building an anchor.** We want the greedy algorithm to *not* move the vertices into the other partition, as shown in Fig. 1.28.

4 **Adding the auxiliary structure.**

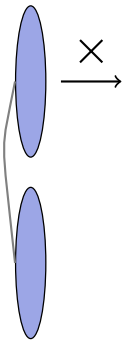


Figure 1.28: An anchor for maxcut problem. We do not want the upper vertices to be moved to the other partition.

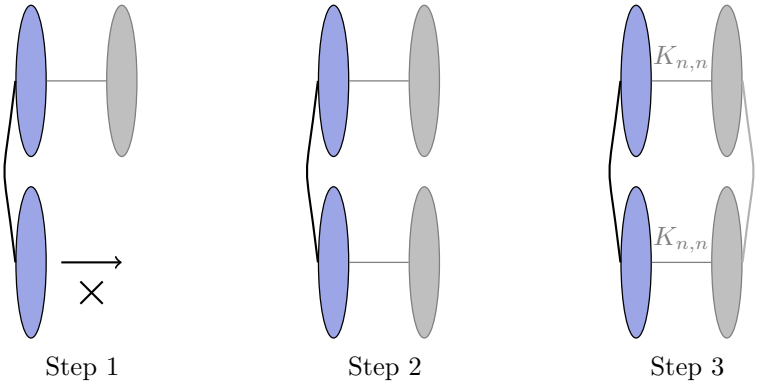


Figure 1.29: We use the anchor as a starting point and add the auxiliary structure in three steps. We get a tight counterexample with goodness ratio 2.

We proceed in three steps.

1. In order to prevent the upper vertices of the anchor from being moved to the other side, we attach a independent graph I_n (least discrepancy graph). However, this is not enough since the greedy algorithm could now move the lower vertices of the anchor (Fig. 1.29).

2. In order to prevent the lower vertices from being moved, we attach another I_n. This gives us a counterexample with goodness ratio 1.5.

3. We can fine tune the counterexample by adding edges between the two attached I_ns. This improves the goodness ratio to 2 which is the best possible ratio.

1.6.5 Maxflow

Optimization Problem. We are given a graph $G = (V, E)$ with two designated vertices as source s and target t. Two paths are said to be *edge-disjoint* if they have no edge in common. Find the maximum number of edge-disjoint paths between s and t.

Greedy Algorithm. Intuitively, we can get more paths if each path is short. A greedy strategy would be to pick the paths in the order of increasing path lengths.

Algorithm 1.6.5: MAXFLOW(G)

Input: A graph $G = (V, E)$ and two vertices s and t belonging to V.

Output: A collection of disjoint paths from s to t.

repeat

1 : Pick the shortest path P from s to t.

2 : Remove all the edges in P.

until vertex s is disconnected from t.

Constructing a counterexample

1 **Weak algorithm.** The weak algorithm works like the greedy algorithm, except that instead of picking the shortest path from s to t it picks *any* path from s to t at each step.

2 **Weak counterexample.** It is easy to imagine a graph where picking a long path disrupts a lot of short disjoint paths. For example, in Fig. 1.30, source s is connected to k vertices. There exists a flow of size k, but the weak algorithm could pick the long path and get only a flow of size 1. This graph has discrepancy of $O(n)$.

3 **Building an anchor.** We want the greedy algorithm to mimic the weak algorithm. This would happen if the dashed short path in the weak counterexample were somehow longer than the longest path. In Fig. 1.30 (b), the dashed lines should be longer than the solid line.

4 **Adding the auxiliary structure.** We replace the dashed lines in the anchor with long paths of size $2k + 3$. This gives us a greedy counterexample (Fig. 1.31). There are $O(k^2)$ vertices in the graph and the maximum flow possible is k. But the greedy algorithm picks one long path and gives a flow of size 1. Hence, the goodness ratio of the counterexample is $O(\sqrt{n})$, which is the best possible.



Figure 1.30: A weak counterexample for the max-flow problem. A flow of $O(n)$ units exists but the weak algorithm finds a flow of only 1 unit.

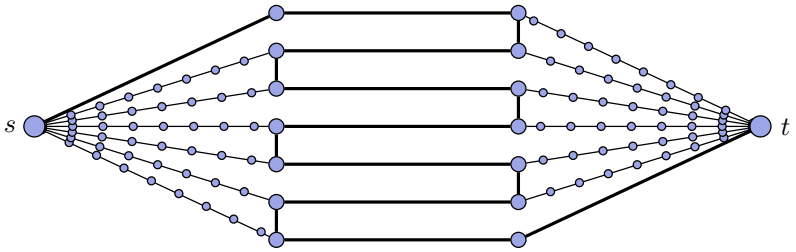


Figure 1.31: A tight counterexample for the greedy max-flow algorithm. The counterexample has goodness ratio of $O(\sqrt{n})$.

1.6.6 Feedback Vertex Set

Optimization Problem. The feedback vertex set problem(FVS) asks for the minimum set of vertices one should remove from a graph in order to make it acyclic. More precisely, given a graph $G = (V, E)$, find a set $F \subseteq V$ such that $G \setminus F$ is acyclic and the size of F is minimum. An example is shown in Fig. 1.32.

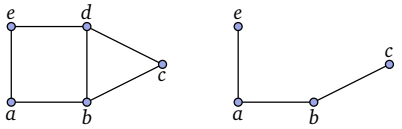


Figure 1.32: We can make the above graph acyclic by just removing vertex d.

Greedy Algorithm. We would like to pick vertices whose removal disrupts a large number

of cycles.

Algorithm 1.6.6: FVS(G)

Input: Graph G

Output: A feedback vertex set F

Initially set F is empty.

1 : **if** graph G has no cycles

report F as the output.

Let u be the vertex in G that in contained

in maximum number of cycles.

2 : Add vertex u to set F

3 : Remove u from G to get a subgraph G'

4 : Recurse on G': FVS(G').

Constructing a counterexample

1 **Weak algorithm.** The greedy choice is to remove the vertex which is contained in maximum number of cycles. The weak algorithm removes any vertex contained in some cycle.

2 **Weak counterexample.** Consider a *flower* with a central vertex that can be removed to make the graph acyclic. The weak algorithm might end up picking one vertex from each cycle.

3 **Building an anchor.** The anchor corresponding to the flower has outer vertices with high degree and the central vertex with low degree.

4 **Adding the auxiliary structure.** We need the outer vertices to be contained in more cycles than the central vertex. Attaching three K_ns as an auxiliary structure serves the purpose. Adding edges between each outer vertex and every vertex from K_ns gives a greedy counterexample with approximation ration 4/3 (Fig. 1.33).

1.6.7 Minimum Dominant Set Problem

Optimization Problem. We are given a graph $G = (V, E)$ as input. A vertex set S is called *dominant* if every vertex in G either belongs to S or is a neighbor of a vertex in S. The

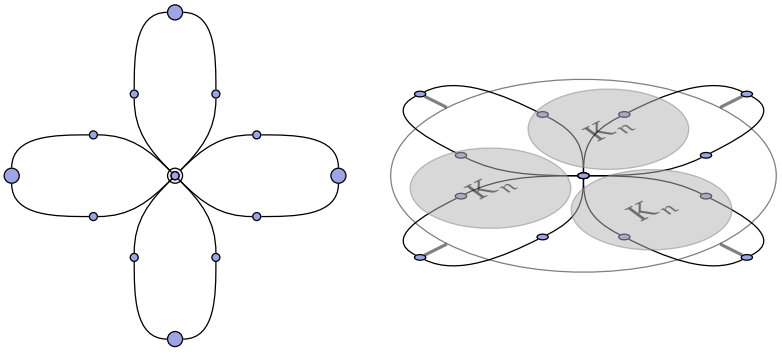


Figure 1.33: Weak counterexample and a greedy counterexample for the feedback vertex set problem.

minimum dominant set problem is to find a dominant set S of minimum size.

Greedy Algorithm. The problem is closely related to vertex cover and we follow a similar greedy strategy. A vertex is *covered* if it is in the dominant set or is a neighbor of a vertex in the dominant set. We proceed by picking those vertices which cover the maximum number of vertices at each step.

Algorithm 1.6.7: MINDOM(G)

Input: A graph G

Output: A dominant set $S \subseteq V$ of graph G

Set S is initially empty.

Initially all the vertices are colored white.

repeat

1 : Pick a vertex v which has the maximum number of white neighbors.

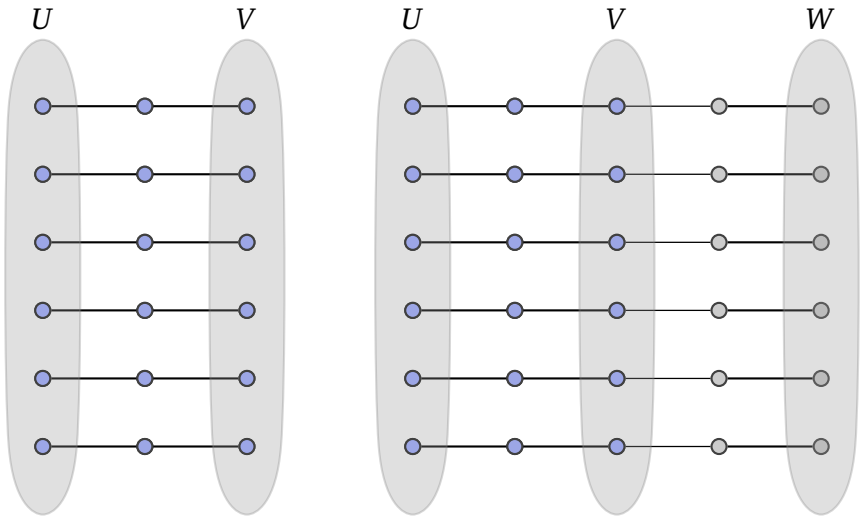
2 : Color v and all it's neighbors black. Add v to S.

until no white colored vertices are left.

Constructing a counterexample

1 **Weak algorithm.** The greedy algorithm picks the vertex with highest number of uncovered neighbors. The weak algorithm picks any vertex and includes it in the dominant set S as long as it is not present in S.

2 **Weak counterexample.** A collection of n paths of length three. The minimum dominant set has size n, but the weak algorithm could pick all the degree one vertices and get a



(a) A weak counterexample for the domination set problem. The optimal solution is to pick the middle vertices of each path. The weak algorithm could pick sets U and V.

(b) A counterexample for the domination set problem with goodness ratio 1.5. The greedy algorithm could pick the set V first (and then U and W). The optimal solution is to pick the second and fourth vertex in each path

Figure 1.34: Construction of counterexample for the MINIMUM DOMINANT SET *problem.*

solution of size $2n$ (Fig. 1.34a).

3 **Building an anchor.** The greedy algorithm will pick vertices in V first if their degree is highest among all the vertices.

4 **Adding the auxiliary structure.** Since the highest degree is 2, it suffices to attach one vertex to each vertex in V. Attaching an collection of n paths of size two gives a greedy counterexample (Fig. 1.34b). The goodness ratio of the graph is 1.5.

Remark. A star is also a weak counterexample with high discrepancy. But unfortunately it cannot be extended to a greedy counterexample.

1.6.8 Minimum Steiner tree

We discuss a problem that is outside the scope of problems we consider. This suggests that the anchor method may be applicable to other kinds of problems too.

Optimization Problem. Given a graph $G = (V, E)$ and a set of *terminal* vertices $P \subseteq V$, the minimum Steiner tree problem (STP) is to find a tree T in G such that the following two

conditions are satisfied:

1. Tree T contains all terminal vertices P.

2. The number of edges in T is minimized.

An example is shown in Fig. 1.35.

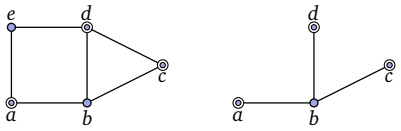


Figure 1.35: The input graph G with terminals $P = \{a, c, d\}$ has a minimum Steiner tree having three edges. Note that the Steiner tree can contain vertices not in set P.

This problem is not within the scope of our problems since the input is a both a graph G and a set of vertices (terminals).

Greedy Algorithm. Notice that the Steiner tree must connect all the terminals using the least number of edges. Since any two terminals must be connected, we try to grow the tree connecting two vertices along their shortest path.

Definition. For a tree $T \in G$ and a vertex $v \in G$, a $v \rightsquigarrow T$ *connecting path* is the shortest path that connects vertex v to some vertex in T.

Algorithm 1.6.8: STP(G)

Input: Graph G and a terminal set P

Output: A minimum steiner tree of G

Initially, the tree T contains only the shortest
path amongst all pairs of terminals in the set P.
repeat
1 : Let u be the terminal not in T that
has the shortest connecting path to T.
2 : Add $u \rightsquigarrow T$ connecting path to T.
until the tree T contains
all the terminals in the set P.
Report tree T as the Steiner tree
for the graph G and terminals P.

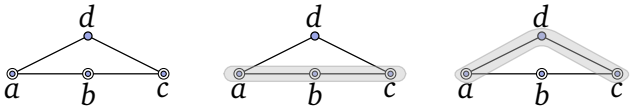


Figure 1.36: A weak counterexample for Steiner tree problem

Constructing a counterexample

1 **Weak algorithm.** We note that the greedy algorithm starts by picking the shortest path between all pairs of terminals. We weaken the algorithm slightly by saying the tree could be initialized with the shortest path between any two terminals.

2 **Weak counterexample.** Let us try to build a counterexample for the weak algorithm. The Steiner tree in the graph having two terminals is the shortest path connecting them. Note that if there are only two terminals the weak algorithm would return the optimal Steiner tree. It is possible to construct a counterexample having 3 terminals as shown in Fig 1.36.

3 **Building an anchor.** The optimal Steiner tree is the path containing 3 terminals. The weak algorithm could initialize tree T by picking the shortest path between a and c going via the non-terminal vertex b. This would result in a Steiner tree containing 3 edges. This serves as our anchor.

4 **Adding the auxiliary structure.** If the greedy algorithm has to take the path adc first, we cannot afford to have distance of 1 between any pair of terminals. We add an edge to move away vertex c so as to ensure all the shortest paths between terminals are 2. This gives a plausible counterexample. We can extend the same idea to obtain a definitive counterexample as shown in Fig. 1.37.

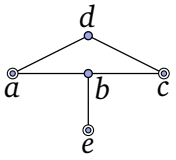


Figure 1.37: A counterexample with goodness ratio 4/3.

1.7 Pilot Experiment

In order to see which heuristics students apply while constructing counterexamples, we did a pilot experiment with four students. All their approaches can be classified into four main

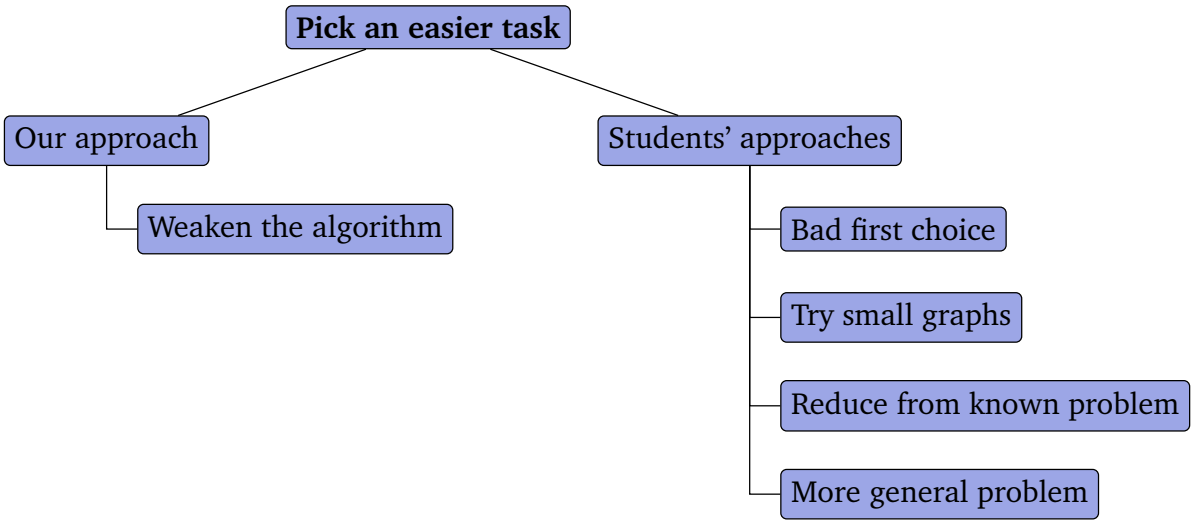


Figure 1.38: Approaches that students tried when constructing counterexamples. The idea of constructing a counterexample to a weak algorithm did not occur to any student.

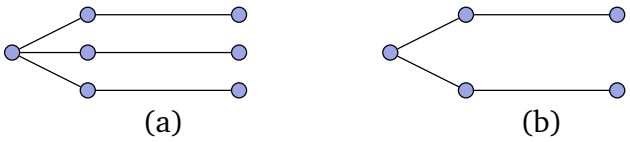


Figure 1.39: A definitive and a plausible counterexample for the vertex cover problem.

categories (Fig.1.38). In this section, we briefly discuss each category by collating their responses. All the students were given the following four problems: MAX INDEPENDENT SET, VERTEX COVER, MATCHING and STEINER TREE.

Force the algorithm to make the wrong choice first. This by far was the most popular heuristic. All the students were able to construct a counterexample for the vertex cover problem using this approach (Fig. 1.39). For the MIS problem, most students wrote a star first and tried to tweak the example such that picking the central vertex leads to a bad choice (Fig. 1.40). A similar approach was tried for matching problem also. So the only successful use of this technique was on the vertex cover problem. We observed that students sometimes failed to acknowledge plausible counterexamples as legitimate counterexamples and seek to get definitive counterexample directly. For example, for the vertex cover problem, three out of four students gave a definitive counterexample and only one student gave a plausible counterexample Fig. 1.39 (b).

Try small graphs. The idea is to check if the algorithm fails for any of the small graphs. This approach seems natural but is not productive for two reasons. Firstly, even with the

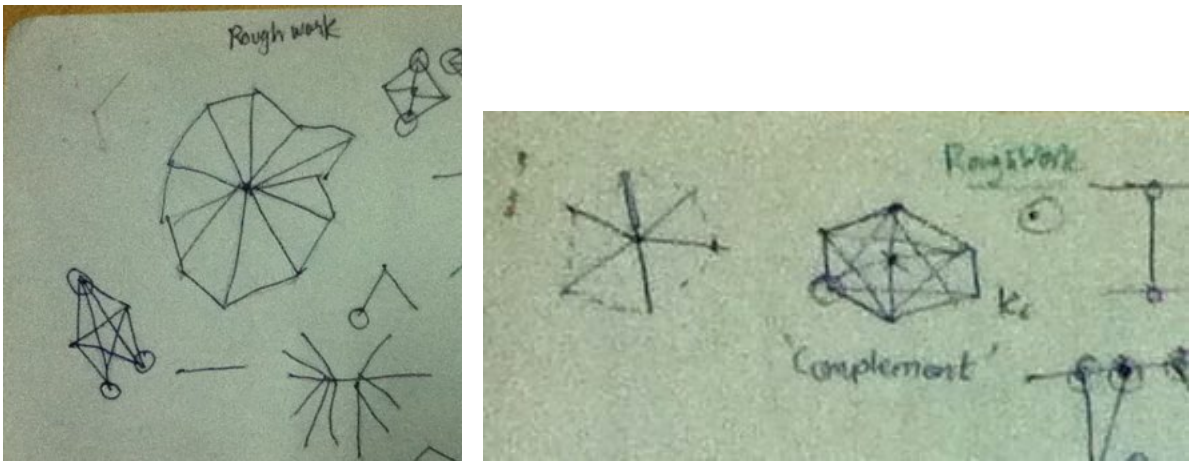


Figure 1.40: Initial rough work by students A and B for the MAX INDEPENDENT SET *problem. Star is an example on which the greedy algorithm performs optimally. The initial reaction in both cases was to try an example that looks very unlike the star example. K_6 ('complement') and K_5 are nearly opposite of a star graph.*

number of nodes not exceeding six, there are many graphs. Unless the student is given a list of small graphs, as shown in Fig. 1.41, it is time-consuming to enlist all the graphs by trail and error. Secondly, many of the greedy algorithms work correctly on small graphs, so the smallest counterexample itself could have more than six nodes. For example, the smallest counterexample for the MIS problem has seven nodes (Fig. 1.8).

Reduce from a known problem. Student C was able to apply this method to construct a counterexample for steiner tree problem. The idea is as follows: since both Steiner tree and vertex cover are NP-complete problems, we can reduce an instance of vertex cover problem to an instance of Steiner tree problem. In this case, it turns out that reduction of the counterexample for vertex cover to Steiner tree instance gives a counterexample for the greedy Steiner tree algorithm (Fig. 1.42a). This seems like a general method that may be applicable for other problems but unfortunately the reduction from one instance to an other itself can be a difficult task.

Generalize the optimization problem. In this method, we see if the greedy algorithm fails for a *harder* optimization problem. If the harder optimization problem is closely related to the original optimization problem, then one might expect to get some idea to solve the original problem. For example, instead of building a Steiner tree with unweighted vertices student D first built a counterexample for the weighted-graph version of the problem and then converted the counterexample into an unweighted graph (Fig. 1.42b).

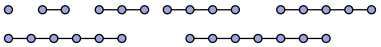
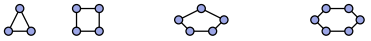
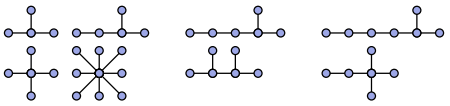
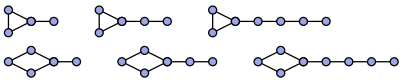
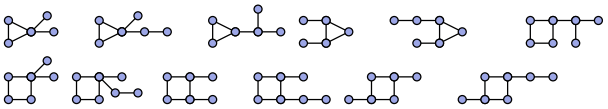
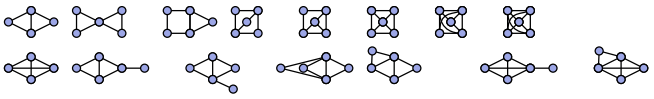
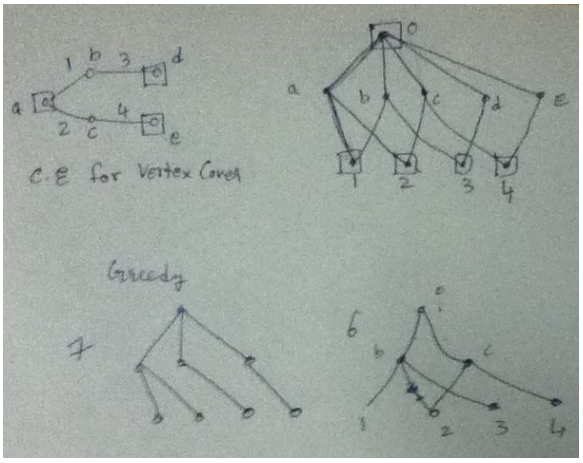
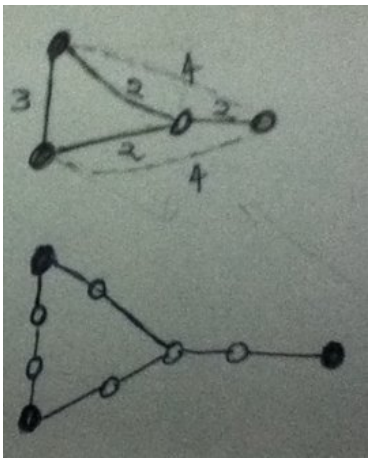
Graph Palette	
Path	
Cycle	
Tree	
1 Cycle, 1 Path	
1 Cycle, 2 Paths	
Small graphs	

Figure 1.41: A list of small graphs.



(a) CE by reduction



(b) CE by weakening the input

Figure 1.42: Successful application of two different heuristics for the Steiner tree problem.

1.8 Discussion

Techniques like greedy, divide-and-conquer, dynamic programming, etc. are quite general and hence the student does not usually know *when* a particular method should be applied. Unlike these techniques, the anchor method is tailor-made for constructing counterexamples for greedy algorithms.

We list some drawbacks of the anchor method:

- Extending the anchor to a counterexample may be hard in some cases. For example, in the max-leaf problem, it is not obvious that we need to swap a star with a complete binary tree.
- One may have to try multiple weak counterexamples before hitting upon the right one that extends to a greedy counterexample.
- Anchor method works only if the weak counterexample gives insight into the structure of a greedy counterexample. It is ineffective for problems like coin changing[1], makespan minimization[3], etc. where this is not the case.

References

[1] S. K. Chang and A. Gill. Algorithmic solution of the change-making problem. *J. ACM*, 17:113–122, January 1970. 32

[2] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to NP-Completeness*. Freeman, San Francisco, CA, USA, 1979. 11

[3] Teofilo Gonzalez and Sartaj Sahni. Open shop scheduling to minimize finish time. *J. ACM*, 23:665–679, October 1976. 32

[4] Sudipto Guha and Samir Khuller. Approximation algorithms for connected dominating sets. *Algorithmica*, 20:374–387, 1996. 18

[5] Jon Kleinberg and Éva Tardos. *Algorithm Design*. Addison Wesley, second edition, 2006. 1, 3, 6

[6] Jagadish M. and Sridhar Iyer. A method to construct counterexamples for greedy algorithms. In *ITiCSE*, pages 238–243, 2012. 1

[7] Shuichi Sakai, Mitsunori Togasaki, and Koichi Yamazaki. A note on greedy algorithms for the maximum weighted independent set problem. *Discrete Appl. Math.*, 126:313–322, March 2003. 3

[8] Douglas B. West. *Introduction to Graph Theory*. Prentice Hall, 2 edition, September 2000. 11